GESP C++ 三級

完整詳解

理解每一步，再寫出答案。

部分	內容
01	考前速查	進制、原碼 / 反碼 / 補碼、位運算、字元、字串和陣列。
02	照片卷	40 題答案與逐題解說；保留原題號。
03	2026 /09、/06、/03	75 題客觀題逐題解析，含原卷措辭與規則辨析。
04	六題編程	解題步驟、樣例推演、常見錯法、帶註釋的完整 C++11 程式。
05	複習索引	照片題號對照三卷題號；錯題記錄表與來源說明。

使用順序：先做《練習題本》，只對答案表；把錯題和沒把握的題圈起來，再讀解析。讀完後闔上詳解，重做同型題。

答案來源：三份 PDF 的答案列與參考程式經逐題核對；本冊解說另行編寫。照片上的手寫答案不作標準答案。原卷有歧義時，明確區分預期答案與嚴格規則。

01 ｜考前速查

主題	要記住的規則	本卷例子
進制位權	右邊第一位是 base^0，每向左一位乘 base。	2102(3)=65；345(8)=E5(16)。
8 位原碼 / 反碼	1 位符號 + 7 位數值；範圍 -127..127。	原碼與反碼都有正零、負零。
8 位補碼	範圍 -128..127；負數可取反加 1。	10110101→181-256=-75。
反碼 vs 補碼	反碼加法要循環進位；補碼保留固定寬度。	不要把兩種進位規則互換。
位運算	&：同為1才1；｜：有1則1；^：不同才1。	x^x=0；x^0=x；~n=-n-1。
優先級	一元 !、~ → <<、>> → & → ^ → |。	(3&7) | (15^(~11))=-5。
字元與數字	'0'=48、'1'=49；字元數字轉值要減 '0'。	(int)'1'=49；'1'-'0'=1。
字串取子串	substr(pos,len) 第二參數是長度。	"GESP2026".substr(4,2)→"20"。
字串刪除	erase(pos) 刪至末尾；erase(pos,1) 刪一字。	"ABCDE" 在 erase(2) 後為 "AB"。
陣列	長度 n，下標 0..n-1；sizeof 是位元組數。	char s[]="GESP" 需要5格。

作答前提與例外

本冊按題目常見設定使用 ASCII、8 位位元組、32 位 int 與補碼。涉及程式語法以 C++11 測試。一般 string 的 size() 數的是 char 元素，不是中文文字個數；若字串內真的存有空字元，它也會計入 size()。

KB 題按原卷 1024 換算；正式單位應區分 kB=1000 B、KiB=1024 B。位元固定寬度的紙筆運算，也不可用來宣稱 C++ 有符號溢位必定環繞。

02｜照片卷答案總表

題號	答案	題號	答案	題號	答案	題號	答案
1	B	2	D	3	D	4	T
5	B	6	T	7	C	8	C
9	F	10	B	11	F	12	C
13	A	14	F	15	B	16	C
17	D	18	B	19	D	20	A
21	D	22	A	23	D	24	A
25	D	26	B	27	T	28	F
29	F	30	F	31	T	32	T
33	T	34	F	35	F	36	T
37	F	38	T	39	T	40	T

先優先訂正這六組

2、4、15、28、30：能否分清原碼與補碼？
5、18、19、25：是否把 !、~、&、|、^ 和移位混淆？
7、9、35、38：整數、字元與編碼是否分清？
8、20、21：empty、substr、erase 的意思是否記準？
10、11、26、39：陣列定義、下標與副本是否掌握？
33、34、40：能否逐步追蹤程式，留意行號和短路？

照片卷｜逐題解說

1 三進制數 2102 轉換成十進制是多少？

答案：B

由右向左的位權是 1、3、9、27。2102(3) = 2×27 + 1×9 + 0×3 + 2×1 = 65。不要把三進制當成二進制。

2 8 位二進制下，十進制數 -15 的補碼是？

答案：D

先寫 +15：00001111；全部取反得 11110000，再加 1 得 11110001。也可用 256 - 15 = 241，再將 241 轉成 8 位二進制。

3 關於計算機的二進制編碼表示，下列說法錯誤的是？

答案：D

n 位有符號補碼範圍是 -2^(n-1) 至 2^(n-1)-1；8 位即 -128 至 127。最大正數少 1，是因為還要表示 0。

辨析｜原卷 B 的「正數全部取反得到反碼」是計算負數補碼的中間步驟；不要把中間結果誤當作該負數的符號原碼取反規則。

4 判斷：求 -11 的補碼，可從 +11 的二進制最右端掃描，遇到第一個 1 後，將左邊各位全部取反。

答案：T（正確）

+11 = 00001011。保留最右邊第一個 1，左側 0000101 取反成 1111010，得到 11110101，即 -11 的 8 位補碼。

5 在 32 位 int、補碼表示的前提下，下列按位取反的說法錯誤的是？

答案：B

在本卷補碼前提下，~n 的數學值等於 -n-1。因此 ~0 = -1；其餘三項分別為 -6、2、-9。注意 ~ 是逐位取反，! 才是邏輯非。

6 判斷：若 int 變數 a 的二進制最低位為 0，則 ((a & 3 & 1) == 0) 為 true。

答案：T（正確）

& 可結合計算，3 & 1 = 1，因此 a & 3 & 1 等於 a & 1，只保留最低位。最低位為 0，整個比較便為 true。

7 若字元變數 _1 的值是字元 '1'，則 (int)_1 的值是？

答案：C

ASCII 中 '0' = 48、'1' = 49。強制轉型讀取編碼值，所以是 49；若要得到數字 1，應使用 _1 - '0'。變數名 _1 中的數字不影響其值。

---

8 關於 string 的成員函式，下列說法錯誤的是？

答案：C

empty() 在字串為空時才回傳 true。D 的完整作用是把取出的字元附加到 s 的末尾，並非只取出而不改動 s。例如 s="ab"、s2="XYZW"，操作後 s="abXYZ"。

---

9 判斷：下列程式中 x 與 y 相等，因為 char 與 int 可以互換。

答案：F（錯誤）

```
char x = '1';
int y = 1;
```

x 儲存的是字元 '1' 的 ASCII 編碼 49；y 儲存整數 1。比較時 x 轉成 int 仍為 49，兩者不相等。char 是否有符號由實作決定，不能一概說 char 能表示 0 至 255。

---

10 以下陣列定義，符合 C++ 語法的是？

答案：B

字元常量 '&' 可以用作整數常量運算式，ASCII 值為 38，所以 b 有 38 個元素。A 的順序應為 int a[10]；C 的 * 不是這裡合法的大小；D 的大小不能用浮點常量。

---

11 判斷：下列程式可把原陣列中所有大於 0 的元素改成 0。

答案：F（錯誤）

```
int a[] = {2, -5, 7, 0, -1};
int n = sizeof(a) / sizeof(a[0]);
for (int num : a) {
    if (num > 0) num = 0;
}
```

int num 是每個元素的副本，改 num 不會改 a。改為 for (int &num : a)，或用下標 a[i] 修改。原碼執行後 a 仍為 {2,-5,7,0,-1}。

---

12 以下問題中，最不適合用枚舉法解決的是？

答案：C

總和有直接公式 n(n+1)/2，得到 1000×1001/2 = 500500。其他問題較適合枚舉候選值後判斷條件。C 也能用循環累加，題目問的是相對最不適合，不是不能使用。

---

13 雞兔同籠：35 個頭、94 隻腳。以下枚舉邏輯最合理的是？

答案：A

頭數已限制雞兔總數為 35，因此只枚舉一種動物就能確定另一種。只需檢查 36 組；答案為雞 23、兔 12。B 有負雞數候選，C 多做大量無用檢查，D 根本沒有求雞兔數。

14 判斷：模擬算法必須逐行複刻手工運算，途中不能加入任何條件判斷。

答案：F（錯誤）

模擬是照題目規則更新狀態，完全可以使用 if、循環、陣列等。例如字元轉換先判斷大小寫再處理，仍是模擬。

15 10110101 是某數的 8 位補碼，該數的十進制值是？

答案：B

最高位為 1，代表負數。把位元先當無符號數：128+32+16+4+1=181；再減 256，得到 -75。或取反加 1 得 01001011=75，再加負號。

16 把相鄰兩個低 4 位的值合併為 1 個位元組。0x1、0x2、0x3、0x4 壓縮後是？

答案：C

把前一個值左移 4 位再按位或：(0x1<<4)|0x2 = 0x12；第二組為 0x34。即 00010010 與 00110100。本卷 B/Q/D/H 分別表示二/八/十/十六進制；這是書面記號，不是 C++ 常量後綴。

17 unsigned char c=0x0F；執行 c=c<<3 後，c 的十進制值為？

答案：D

0x0F=15；15 左移 3 位為 120，二進制由 00001111 變成 01111000。結果在 unsigned char 的範圍內，可安全存回。算式計算前 c 會整數提升，本題仍得到 120。

18 對 32 位有符號 int n，下列關於 ~ 的說法正確的是？

答案：B

逐項代入：~6=-7；~0=-1；~(-2)=1，所以只有 B 正確。這裡把 -n-1 當數學恆等式；不要對最小 int 直接用先算 -n 的程式寫法，以免溢位。

19 計算 3 & 7 | 15 ^ ~11 的結果。

答案：D

優先順序是 ~、&、^、|，原式為 (3&7) | (15^(~11))。3&7=3；~11=-12；15^(-12)=-5；3|(-5)=-5。以低 8 位驗算：00001111 XOR 11110100 = 11111011，再 OR 00000011 仍為 11111011。

---

20 執行下列程式後，sub 的值是？

答案：A

```
string str = "GESP2026";
string sub = str.substr(4, 2);
```

下標從 0 開始：G0 E1 S2 P3 2(4) 0(5) 2(6) 6(7)。substr(4,2) 由下標 4 起取 2 個字元，得到 "20"。第二個參數是長度，不是結束下標。

---

21 關於 string 的成員函式，下列說法錯誤的是？

答案：D

erase(2) 從下標 2 刪到末尾；要只刪 1 個字元，應寫 erase(2,1)。例如 "ABCDE".erase(2) 的效果是留下 "AB"；erase(2,1) 則留下 "ABDE"。

---

22 關於計算機的數據編碼，下列說法正確的是？

答案：A

同一位元模式的值取決於編碼。原碼：符號 1、絕對值 0001101=13，所以 -13；無符號：128+13=141；若當補碼則為 141-256=-115。電腦底層以位元表示，八進制和十六進制是便於人閱讀的表示法。

辨析 | 本卷用 KB=1024 B；現代單位應分清 kB=1000 B、KiB=1024 B。故 D 按試卷慣例判錯，不能當成普遍單位規則。

---

23 關於進制轉換，下列說法不正確的是？

答案：D

345(8)=3×64+4×8+5=229；229=14×16+5，所以是 E5H。E3H=14×16+3=227，兩者相差 2。A、B、C 的展開式均正確。

---

24 按本卷 1 KB=1024 B 的慣例，2 KB 等於多少 bit？

答案：A

先 KB 轉 Byte，再 Byte 轉 bit：2×1024×8=16384 bit。2048 是位元組數，還沒乘每位元組的 8 個 bit。

---

25 下列變數皆為正整數，哪個位運算說法錯誤？

答案：D

例如 5>>1=2，但實數除法 5/2=2.5；被丟棄的低位包含餘數資訊。A 講的是正整數的整數除法，與 D 的精確除法不同。

---

26 以下陣列定義，符合 C++ 語法的是？

答案：B

ASCII 中 'A'=65，可作陣列大小。A 在 C++11 列表初始化中由 double 到 int 屬窄化轉換，即使 3.0 恰為整數也不允許；C 缺大小又缺初始化；D 大小不是整數常量。

---

27 判斷：反碼運算產生最高位進位時，需把進位加回結果最低位。

答案：T (正確)

這叫循環進位。例如 8 位反碼的 5 + (-3)：00000101 + 11111100 = 1 00000001，把溢出的 1 加回低位得 00000010，即 2。要與補碼的處理分開記。

---

28 判斷：8 位補碼表示範圍為 -127 至 +127。

答案：F (錯誤)

補碼範圍是 -128 至 127；10000000 表示 -128。-127 至 127 是 8 位原碼或反碼的數值範圍，它們有兩種零的表示。

---

29 判斷：補碼加法最高位有進位時，需要把它循環加回結果最低位。

答案：F (錯誤)

固定 8 位補碼加法保留低 8 位，丟棄超出寬度的進位，不做循環加回。例如 5+(-3)：00000101+11111101=1 00000010，保留 00000010=2。

辨析 | 位元計算模型與 C++ 有符號溢位要區分：超出 int 可表示範圍的有符號加法不能當作可依賴的環繞結果。

---

30 判斷：8 位二進制原碼的有符號數範圍為 -128 至 +127。

答案：F (錯誤)

原碼把 1 位留給符號，只剩 7 位表示絕對值；最大絕對值為 127，因此範圍是 -127 至 127。10000000 在原碼中是負零，不是 -128。

---

31 判斷：若 int a 滿足 ((a & 1) == 1)，則 a 是奇數。

答案：T（正確）

在本卷補碼前提下，最低位為 1 代表奇數，包含負奇數；不能因此進一步推斷 a 是正數。與九月判斷第 5 題的「正奇數」不同。

---

32 判斷：C++ 的位運算符有不同優先級，使用括號能明確指定分組。

答案：T（正確）

先處理一元 ~、！，再移位 <<、>>，再 &、^、 |。寫 (a&1)==1 最清楚；不要省成 a&1==1 後只靠直覺判斷。括號只決定分組，並不能解決所有副作用求值順序問題。

---

33 判斷：下列程式若能正確執行，會輸出 GESP。

答案：T（正確）

```
int main() {
    string name = "GESP";
    cout << name[false] << name[true]
         << name[1 << 1] << name[7 >> 1] << endl;
}
```

false 轉成下標 0，true 轉成 1；1<<1=2，7>>1=3。因此依序輸出 name[0]、[1]、[2]、[3]，即 GESP。題幹已假定能正確執行，所需標頭和命名空間不在此判斷範圍。

---

34 判斷：把下列包含行首數字的所有文字輸入編譯器，可正常編譯執行。

答案：F（錯誤）

```
1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     cout << "Hello world!" << endl;
5     return 0;
6 }
```

行首 1、2、3… 是展示用行號，不能作為程式文字一起輸入。原卷把行號也打進程式，因此不能正常編譯。刪掉這些行首數字才是題目想表達的正常程式。

---

35 判斷：檢查 int 數字 s[i] 是否在 0 至 9，可寫 s[i]>='0' && s[i]<='9'。

答案：F（錯誤）

若 s[i] 儲存整數數值，應寫 s[i]>=0 && s[i]<=9。'0' 和 '9' 是字元編碼 48 與 57，原條件實際測試 48 至 57。只有儲存字元編碼時才用帶引號的比較。

---

36 判斷：枚舉可以在保證不漏解的前提下縮小範圍，不必遍歷所有想像得到的值。

答案：T（正確）

例如雞兔同籠只枚舉雞數 0..35，兔數由總頭數推出；檢查質數只須測試到平方根。縮小範圍的理由必須成立，不能憑猜測排除可能的解。

---

37 判斷：((0xf0 + 0x15) == 255) 的值為 true。

答案：F（錯誤）

0xF0=240，0x15=21，總和為 261，即 0x105，不是 255。255 的十六進制才是 0xFF。這裡是一般 int 計算，不會因寫了兩位十六進制就截成 8 位。

---

38 判斷：已知 'A' 的 ASCII 為 0x41，則 'm' 的 ASCII 八進制為 155。

答案：T（正確）

'A'=65，'a'=97，'m' 比 'a' 大 12，所以為 109。109=1×64+5×8+5，八進制為 155；C++ 八進制常量可寫 0155。

---

39 判斷：int arr[]={1,2,3}；若 int 佔 4 Byte，sizeof(arr) 為 12。

答案：T（正確）

實際陣列有 3 個 int，總大小為 3×4=12 Byte。元素數是 sizeof(arr)/sizeof(arr[0])。此結論只適用於這裡仍是陣列的 arr，不能照搬到函式參數中已退化的指標。

---

40 判斷：下列程式會輸出 98。

答案：T（正確）

```
#include <iostream>
using namespace std;
int main() {
    int a = 'a', b = 'b';
    if (a > 0 || b++ > 0) {
    }
    cout << b;
}
```

ASCII 中 a=97、b=98；a>0 為 true。邏輯或 || 在左側為 true 時短路，右邊 b++>0 不執行，因此 b 保持 98。不要把 || 誤當成位運算 |。

---

03 | 2026 / 09 客觀題答案

以下題號對應該月份原卷，不是照片卷題號。

選擇題

題號	答案	題號	答案	題號	答案	題號	答案
1	D	2	D	3	A	4	C
5	B	6	C	7	D	8	C
9	A	10	A	11	D	12	C
13	D	14	B	15	B		

判斷題

題號	答案	題號	答案	題號	答案	題號	答案
1	F	2	F	3	F	4	F
5	F	6	F	7	F	8	T
9	T	10	F				

編程題

1. 公共二進制位：給 n 個非負整數，2 ≤ n ≤ 20000，0 ≤ a[i] ≤ 10^9。求在所有數中都為 1 的二進制位共有幾個。

2. 分割字串：輸入一行長度不超過 1000 的文字，可含字母、數字和空格，且不以空格結尾。第一次以 1 個空格分割右側文字，第二次以連續 2 個，第三次以連續 3 個，以此類推；只丟棄本次所需的空格。

2026 / 09 | 選擇題解析

選 1 反碼的最高位進位應丟棄

答案：D

反碼加法需要循環進位，把最高位進位加回最低位；直接丟棄是本卷固定寬度補碼運算的做法。A 的正數 5 三種編碼都為 00000101，C 的原碼敘述亦正確。

辨析 | B 的 KB=1024 B 是本卷慣例；正式單位應區分 kB 與 KiB，不能視為所有領域的通則。

選 2 關於進制轉換，下列說法不正確的是？

答案：D

345(8)=3×64+4×8+5=229；229=14×16+5，所以是 E5H。E3H=14×16+3=227，兩者相差 2。A、B、C 的展開式均正確。

選 3 for 的條件寫成 i=0

答案：A

for(init; condition; update) 的中間是條件。這裡 i=0 是合法賦值運算式，其值為 0（false），第一次檢查就不進循環，所以 x 仍為 1。第三部分 i<=5 雖是合法但無作用的比較，本題根本不會執行到它。

選 4 8 位補碼最大值寫成 128

答案：C

8 位補碼的最大值為 127，最小值為 -128，所以 C 的正端 128 不成立。正數編碼不變；負數可對其正數位元全部取反再加 1，或使用從右端保留到第一個 1 的技巧。

選 5 x=x&(x-1) 計數

答案：B

每次 x&(x-1) 都消去 x 最右邊的一個 1。2026=1024+512+256+128+64+32+8+2，其二進制為 11111101010，含 8 個 1，因此循環執行 8 次。

選 6 有符號左移溢出結果確定

答案：C

不能把有符號左移超出範圍當成可依賴的固定結果。依本卷 C++11 作答慣例，這項敘述錯；實務上要避免非法移位，並用適當無符號型別處理位元。末尾兩位為 00 的非負二進制數是 4 的倍數。

選 7 計算 3 & 7 | 15 ^ ~11 的結果。

答案：D

優先順序是 ~、&、^、|，原式為 (3&7) | (15^(~11))。3&7=3；~11=-12；15^(-12)=-5；3|(-5)=-5。以低 8 位驗算：00001111 XOR 11110100 = 11111011，再 OR 00000011 仍為 11111011。

選 8 異或加密後如何解密

答案：C

對每個字元再異或相同 key： (x^key)^key=x^(key^key)=x。或、與、移位通常會丟失原資訊，不能保證恢復原字元。

選 9 用最大字元判斷可能進制

答案：A

在合法數碼字元的前提下，用 max<='1'、<='7'、<='9'、<='F'，所以選 A。B 把字元編碼與整數 1、7、9 混比；C、D 也沒有正確字元常量。

辨析 | 與六月選擇第 15 題同型。若輸入允許任意符號，還需額外逐字驗證，不能只看最大字元。

選 10 位掩碼權限組合

答案：A

新增權限用 |，清除某權限用 &~。mask 從 0000 開始：加讀得 0001；加寫得 0011；清讀得 0010；加刪得 1010，最後十進制為 10。不能用 |~READ 清除，否則會把大量其他位設為 1。

選 11 下列變數皆為正整數，哪個位運算說法錯誤？

答案：D

例如 5>>1=2，但實數除法 5/2=2.5；被丟棄的低位包含餘數資訊。A 講的是正整數的整數除法，與 D 的精確除法不同。

選 12 !a & b << c ^ d | e

答案：C

a=0、b=2、c=11、d=7、e=5。分組為 ((!a)&(b<<c))^d|e。!0=1；2<<11=4096；1&4096=0；0^7=7；7|5=7。不要誤把 & 放在 << 前計算。

選 13 關於 string 的成員函式，下列說法錯誤的是？

答案：D

erase(2) 從下標 2 刪到末尾；要只刪 1 個字元，應寫 erase(2,1)。例如 "ABCDE".erase(2) 的效果是留下 "AB"；erase(2,1) 則留下 "ABDE"。

選 14 以下陣列定義，符合 C++ 語法的是？

答案：B

ASCII 中 'A'=65，可作陣列大小。A 在 C++11 列表初始化中由 double 到 int 屬窄化轉換，即使 3.0 恰為整數也不允許；C 缺大小又缺初始化；D 大小不是整數常量。

選 15 兩輪辨認 1000 個酒缸

答案：B

每隻實驗動物可區分三種結果：第一輪後、第二輪後、兩輪後仍存活，對應三進制一位。m 隻最多編碼 3^m 個酒缸。3^6=729<1000，3^7=2187>=1000，所以至少 7 隻。題目是在考三進制資訊編碼。

2026 / 09 | 判斷題解析

判 1 判斷：補碼加法最高位有進位時，需要把它循環加回結果最低位。

答案：F（錯誤）

固定 8 位補碼加法保留低 8 位，丟棄超出寬度的進位，不做循環加回。例如 5+(-3)：00000101+11111101=1 00000010，保留 00000010=2。

辨析 | 位元計算模型與 C++ 有符號溢位要區分：超出 int 可表示範圍的有符號加法不能當作可依賴的環繞結果。

判 2 求 -13 補碼時，把第一個 1 右側取反

答案：F（錯誤）

方向說反了。應從 +13=00001101 的最右端保留到第一個 1，再取反其左側各位，得到 11110011。不是保留左側、反轉右側。

判 3 判斷：8 位二進制原碼的有符號數範圍為 -128 至 +127。

答案：F（錯誤）

原碼把 1 位留給符號，只剩 7 位表示絕對值；最大絕對值為 127，因此範圍是 -127 至 127。10000000 在原碼中是負零，不是 -128。

判 4 判斷：下列程式可把原陣列中所有大於 0 的元素改成 0。

答案：F（錯誤）

int num 是每個元素的副本，改 num 不會改 a。改為 for (int &num : a)，或用下標 a[i] 修改。原碼執行後 a 仍為 {2,-5,7,0,-1}。

判 5 (a&1)==1 表示正奇數

答案：F（錯誤）

只能知道是奇數，不能判斷正負。在本卷補碼表示下，-3 的最低位也是 1，所以 -3&1=1，卻不是正數。

判 6 char str[4]="GESP" 會自動分配 5 Byte

答案：F (錯誤)

陣列指定大小是 4，編譯器不會自動增加為 5。字串常量含 G、E、S、P 與 '\0' 共 5 個字元，此 C++ 初始化不合法。可改成 char str[5]="GESP" 或 char str[]="GESP"。

判 7 用 str[i]==0 || str[i]==1 檢查字元 '0' 或 '1'

答案：F (錯誤)

字元 '0'、'1' 的 ASCII 分別為 48、49，與整數 0、1 不同。應寫 str[i]=='0' || str[i]=='1'。

判 8 判斷：下列程式會輸出 98。

答案：T (正確)

ASCII 中 a=97、b=98；a>0 為 true。邏輯或 || 在左側為 true 時短路，右邊 b++>0 不執行，因此 b 保持 98。不要把 || 誤當成位運算 |。

判 9 判斷：枚舉可以在保證不漏解的前提下縮小範圍，不必遍歷所有想像得到的值。

答案：T (正確)

例如雞兔同籠只枚舉雞數 0..35，兔數由總頭數推出；檢查質數只須測試到平方根。縮小範圍的理由必須成立，不能憑猜測排除可能的解。

判 10 i<n 的流程圖能累加 a[0] 至 a[n]

答案：F (錯誤)

i 從 0 開始，條件為 i<n，因此只處理 a[0]..a[n-1]，共 n 個元素，不包含 a[n]。若題意是長度 n 的陣列，流程本身是標準寫法；錯在題幹宣稱包含 a[n]。

---

03 | 2026 / 06 客觀題答案

以下題號對應該月份原卷，不是照片卷題號。

選擇題

題號	答案	題號	答案	題號	答案	題號	答案
1	A	2	B	3	C	4	B
5	A	6	C	7	C	8	A
9	B	10	D	11	D	12	B
13	C	14	B	15	B		

判斷題

題號	答案	題號	答案	題號	答案	題號	答案
1	T	2	T	3	F	4	T
5	T	6	F	7	T	8	F
9	F	10	F				

編程題

1. 加密：先輸入 n 個 0..9 的數字，再輸入 10 個數字作為對應表；n 不超過 20000。把每個輸入數字替換成密碼本指定的數字。

2. 字元轉換：輸入 n 和長度為 n 的字串，字元只可能為大小寫英文字母或數字，n ≤ 1000。大小寫互換，數字改為 *。

2026 / 06 | 選擇題解析

選 1 關於計算機的數據編碼，下列說法正確的是？

答案：A

同一位元模式的值取決於編碼。原碼：符號 1、絕對值 0001101=13，所以 -13；無符號：128+13=141；若當補碼則為 141-256=-115。電腦底層以位元表示，八進制和十六進制是便於人閱讀的表示法。

辨析 | 本卷用 KB=1024 B；現代單位應分清 kB=1000 B、KiB=1024 B。故 D 按試卷慣例判錯，不能當成普遍單位規則。

選 2 廠商標示 1 TB 硬碟

答案：B

十進制 1 TB = 1000^4 Byte。轉為每單位 1024^3 Byte：1000^4/1024^3 ≈ 931.32，故選約 931 的 B。容量沒有真的少掉，而是單位基準不同。

辨析 | 原卷選項把結果寫成 GB；嚴格應為約 931 GiB，或 1000 GB。A 把除數寫成 1024+1024+1024，不是正確單位換算。

選 3 把相鄰兩個低 4 位的值合併為 1 個位元組。0x1、0x2、0x3、0x4 壓縮後是？

答案：C

把前一個值左移 4 位再按位或：(0x1<<4)|0x2 = 0x12；第二組為 0x34。即 00010010 與 00110100。本卷 B/Q/D/H 分別表示二/八/十/十六進制；這是書面記號，不是 C++ 常量後綴。

選 4 反碼與補碼的範圍

答案：B

原卷 B 把 n 位補碼範圍寫成對稱的 -(2^(n-1)-1) 至 +(2^(n-1)-1)。補碼負端應是 -2^(n-1)，例如 8 位為 -128..127。反碼才是 -127..127，並有正零和負零。

辨析 | 題幹同時提反碼與補碼，B 沒有清楚指定編碼；給定答案採補碼解讀。若 B 專指原碼或反碼，其對稱範圍成立，因此有措辭歧義。

選 5 GESP 按指定運算與 KEY=2026 加密

答案：A

依題意從密鑰低位依序取 6、2、0、2：'G'|6=71|6=71='G'；'E'-2=67='C'；'S'^0='S'；'P'+2=82='R'，得到 GCSR。不要自行把運算順序或密鑰數字順序顛倒。

選 6 左移一定等於乘 2 的冪嗎

答案：C

沒有溢位且移位合法的前提下，正數左移 k 位可視為乘 2^k。一旦丟棄高位或涉及不合法的有符號運算，就不能說結果仍等於數學乘積，故 C 錯。其餘說法按題目慣例成立。

選 7 字元陣列與 strcpy/strcat

答案：C

A："GESP" 的陣列大小為 5，而 strlen 為 4。B：char str1[4]="GESP" 沒空間放 '\0'，C++ 不合法。C：4 格陣列放不下 HELLO,GESP 加結尾的 10 個字元，發生越界。D：dest 本來就沒有結尾空字元且空間不足，不能安全 strcat。

選 8 按本卷 1 KB=1024 B 的慣例，2 KB 等於多少 bit？

答案：A

先 KB 轉 Byte，再 Byte 轉 bit：2×1024×8=16384 bit。2048 是位元組數，還沒乘每位元組的 8 個 bit。

選 9 對 32 位有符號 int n，下列關於 ~ 的說法正確的是？

答案：B

逐項代入：~6=-7；~0=-1；~(-2)=1，所以只有 B 正確。這裡把 -n-1 當數學恆等式；不要對最小 int 直接用先算 -n 的程式寫法，以免溢位。

選 10 關於計算機的二進制編碼表示，下列說法錯誤的是？

答案：D

n 位有符號補碼範圍是 -2^(n-1) 至 2^(n-1)-1；8 位即 -128 至 127。最大正數少 1，是因為還要表示 0。

辨析 | 原卷 B 的「正數全部取反得到反碼」是計算負數補碼的中間步驟；不要把中間結果誤當作該負數的符號原碼取反規則。

選 11 正整數位運算的錯誤敘述

答案：D

D 聲稱左移超出範圍仍符合乘法規律，錯。A 用 &1 判奇偶；B 的 &0xFF 保留低 8 位；C 的 num&b 只會清除 num 中的位元，因此在非負整數範圍不大於 num。

選 12 !a << b & c ^ d | e

答案：B

給定 a=7、b=3、c=14、d=15、e=8。依優先級分組為 (((!a)<<b)&c)^d|e。!7=0；0<<3=0；0&14=0；0^15=15；15|8=15。

選 13 關於 string 的成員函式，下列說法錯誤的是？

答案：C

empty() 在字串為空時才回傳 true。D 的完整作用是把取出的字元附加到 s 的末尾，並非只取出而不改動 s。例如 s="ab"、s2="XYZW"，操作後 s="abXYZ"。

選 14 以下陣列定義，符合 C++ 語法的是？

答案：B

字元常量 '&' 可以用作整數常量運算式，ASCII 值為 38，所以 b 有 38 個元素。A 的順序應為 int a[10]；C 的 * 不是這裡合法的大小；D 的大小不能用浮點常量。

選 15 用最大字元判斷可能進制

答案：B

在字元只來自 0..9、A..F 的前提下，分別檢查 max<='1'、max<='7'、max<='9'、max<='F'。要包含邊界，所以是 <=；比較的是字元，1、7、9 要加單引號。

辨析｜題幹未清楚排除其他符號；一般輸入驗證不能只比較最大字元，例如 / 也可能通過比較。完整程式須逐字檢查合法數碼。

2026 / 06 | 判斷題解析

判 1 判斷：反碼運算產生最高位進位時，需把進位加回結果最低位。

答案：T（正確）

這叫循環進位。例如 8 位反碼的 5 + (-3)：00000101 + 11111100 = 1 00000001，把溢出的 1 加回低位得 00000010，即 2。要與補碼的處理分開記。

判 2 判斷：求 -11 的補碼，可從 +11 的二進制最右端掃描，遇到第一個 1 後，將左邊各位全部取反。

答案：T（正確）

+11 = 00001011。保留最右邊第一個 1，左側 0000101 取反成 1111010，得到 11110101，即 -11 的 8 位補碼。

判 3 判斷：8 位補碼表示範圍為 -127 至 +127。

答案：F（錯誤）

補碼範圍是 -128 至 127；10000000 表示 -128。-127 至 127 是 8 位原碼或反碼的數值範圍，它們有兩種零的表示。

判 4 質數檢查可縮小到平方根

答案：T（正確）

若 n=ab 且 a、b 都大於 sqrt(n)，乘積會大於 n，因此合數一定有一個因數不超過 sqrt(n)。可枚舉 i=2 且 i*i<=n；另須先排除 n<2。避免大數平方溢位時可用 i<=n/i。

判 5 判斷：若 int a 滿足 ((a & 1) == 1)，則 a 是奇數。

答案：T（正確）

在本卷補碼前提下，最低位為 1 代表奇數，包含負奇數；不能因此進一步推斷 a 是正數。與九月判斷第 5 題的「正奇數」不同。

判 6 CCF 的十進制為 3269

答案：F（錯誤）

0xCCF=12×256+12×16+15=3279，而非 3269。二進制 1100 1100 1111 正確；分成三位一組 110 011 001 111 得八進制 6317，也正確。整題仍因十進制一項錯而判錯。

判 7 判斷：下列程式若能正確執行，會輸出 GESP。

答案：T（正確）

false 轉成下標 0，true 轉成 1；1<<1=2，7>>1=3。因此依序輸出 name[0]、[1]、[2]、[3]，即 GESP。題幹已假定能正確執行，所需標頭和命名空間不在此判斷範圍。

判 8 判斷：把下列包含行首數字的所有文字輸入編譯器，可正常編譯執行。

答案：F（錯誤）

行首 1、2、3… 是展示用行號，不能作為程式文字一起輸入。原卷把行號也打進程式，因此不能正常編譯。刪掉這些行首數字才是題目想表達的正常程式。

判 9 判斷：模擬算法必須逐行複刻手工運算，途中不能加入任何條件判斷。

答案：F（錯誤）

模擬是照題目規則更新狀態，完全可以使用 if、循環、陣列等。例如字元轉換先判斷大小寫再處理，仍是模擬。

判 10 判斷：檢查 int 數字 s[i] 是否在 0 至 9，可寫 s[i]>='0' && s[i]<='9'。

答案：F（錯誤）

若 s[i] 儲存整數數值，應寫 s[i]>=0 && s[i]<=9。'0' 和 '9' 是字元編碼 48 與 57，原條件實際測試 48 至 57。只有儲存字元編碼時才用帶引號的比較。

---

03 | 2026 / 03 客觀題答案

以下題號對應該月份原卷，不是照片卷題號。

選擇題

題號	答案	題號	答案	題號	答案	題號	答案
1	C	2	B	3	C	4	A
5	D	6	B	7	B	8	D
9	B	10	A	11	B	12	B
13	C	14	A	15	D		

判斷題

題號	答案	題號	答案	題號	答案	題號	答案
1	F	2	F	3	T	4	T
5	T	6	T	7	T	8	F
9	F	10	T				

編程題

1. 二進制回文串：輸入 $1 \le n \le 100000$，計算 1 至 n 中，無前導零的二進制表示為回文的整數數量。

2. 凱撒密碼：給定已知明文、對應密文、待破解密文，三者均為大寫字母且長度不超過 1000。求待破解文字的明文。

2026 / 03 | 選擇題解析

選 1 若字元變數 _1 的值是字元 '1'，則 (int)_1 的值是？

答案：C

ASCII 中 '0' = 48、'1' = 49。強制轉型讀取編碼值，所以是 49；若要得到數字 1，應使用 _1 - '0'。變數名 _1 中的數字不影響其值。

選 2 三次異或交換 a 與 b

答案：B

設原來 a=A、b=B。第一步 a=A^B；第二步 b=(A^B)^B=A；第三步 a=(A^B)^A=B，所以兩值互換。條件是 a、b 是不同的儲存位置；相同物件不能用此法交換。

選 3 字元陣列 str1 與 str2

答案：C

str1="Hello" 自動包含末尾空字元，共 6 格；str2={'H','e','l','l','0'} 只有 5 格，而且最後是數字字元 '0'，不是小寫 o，更不是 '\0'。兩者不同。把沒有終止空字元的 str2 交給 cout 當 C 字串讀取，可能越界，屬未定義行為。

辨析 | 原題稱「正確的程式段」不夠嚴謹。選 C 不代表整段輸出操作安全；B「每次輸出必不同」也不是未定義行為的正確描述。

選 4 (x++) + (++x)

答案：A

+ 兩邊都修改同一個 x，且這兩次修改沒有規定先後順序，屬未定義行為。不能指定答案為 20、21 或 22；一次實測得到某個數也不構成規則。括號只分組，沒有建立兩邊的先後順序。

選 5 8 位二進制下，十進制數 -15 的補碼是？

答案：D

先寫 +15：00001111；全部取反得 11110000，再加 1 得 11110001。也可用 256 - 15 = 241，再將 241 轉成 8 位二進制。

選 6 三進制數 2102 轉換成十進制是多少？

答案：B

由右向左的位權是 1、3、9、27。2102(3) = 2×27 + 1×9 + 0×3 + 2×1 = 65。不要把三進制當成二進制。

選 7 10110101 是某數的 8 位補碼，該數的十進制值是？

答案：B

最高位為 1，代表負數。把位元先當無符號數：128+32+16+4+1=181；再減 256，得到 -75。或取反加 1 得 01001011=75，再加負號。

選 8 unsigned char c=0x0F；執行 c=c<<3 後，c 的十進制值為？

答案：D

0x0F=15；15 左移 3 位為 120，二進制由 00001111 變成 01111000。結果在 unsigned char 的範圍內，可安全存回。算式計算前 c 會整數提升，本題仍得到 120。

選 9 在 32 位 int、補碼表示的前提下，下列按位取反的說法錯誤的是？

答案：B

在本卷補碼前提下，~n 的數學值等於 -n-1。因此 ~0 = -1；其餘三項分別為 -6、2、-9。注意 ~ 是逐位取反，! 才是邏輯非。

選 10 執行下列程式後，sub 的值是？

答案：A

下標從 0 開始：G0 E1 S2 P3 2(4) 0(5) 2(6) 6(7)。substr(4,2) 由下標 4 起取 2 個字元，得到 "20"。第二個參數是長度，不是結束下標。

選 11 sizeof 求陣列元素數

答案：B

arr 有 6 個 int。sizeof(arr) 為整個陣列的位元組數，除以 sizeof(arr[0]) 得到 6。不要把元素數與記憶體位元組數混淆。

選 12 char s[10]="abcde"；a-b

答案：B

sizeof(s)/sizeof(s[0])=10，strlen(s)=5。s 內有 5 個可見字元，後面包含終止空字元和補零，但 strlen 只數到第一個 '\0' 前。因此 10-5=5。

選 13 以下問題中，最不適合用枚舉法解決的是？

答案：C

總和有直接公式 n(n+1)/2，得到 1000×1001/2 = 500500。其他問題較適合枚舉候選值後判斷條件。C 也能用循環累加，題目問的是相對最不適合，不是不能使用。

選 14 雞兔同籠：35 個頭、94 隻腳。以下枚舉邏輯最合理的是？

答案：A

頭數已限制雞兔總數為 35，因此只枚舉一種動物就能確定另一種。只需檢查 36 組；答案為雞 23、兔 12。B 有負雞數候選，C 多做大量無用檢查，D 根本沒有求雞兔數。

選 15 小寫字母 +3，超出 z 要環繞

答案：D

應先檢查 str[i]+3>'z'，超出時使用 'a'+(str[i]+3-'z')-1。例：'x'+3 超出 z 1 格，回到 a。若未超出才在 else 裡加 3；不能只減 26 而漏了加 3。

2026 / 03 | 判斷題解析

判 1 arr[3] 為 0，而且 arr[5] 是合法下標

答案：F（錯誤）

int arr[5]={1,2,3} 會把其餘元素補 0，所以 arr[3]=0；但合法下標只有 0..4，arr[5] 越界。合取敘述只要一半錯，整題就是錯。

判 2 double arr[10] 未手動初始化就全為 0

答案：F（錯誤）

不能不分宣告位置就認定全為 0。區域自動陣列未初始化時不可當作全零；全域或 static 陣列則會零初始化。要明確全零可寫 double arr[10]={}。

判 3 判斷：int arr[]={1,2,3}；若 int 佔 4 Byte，sizeof(arr) 為 12。

答案：T（正確）

實際陣列有 3 個 int，總大小為 3×4=12 Byte。元素數是 sizeof(arr)/sizeof(arr[0])。此結論只適用於這裡仍是陣列的 arr，不能照搬到函式參數中已退化的指標。

判 4 流程圖求 1+2+…+10

答案：T（正確）

初始化 sum=0、i=1；只要 i<=10，就先 sum+=i，再 i++。依序加入 1..10，i=11 時結束，輸出 55。

判 5 流程圖計算 5 對 2 取餘

答案：T（正確）

a=5、b=2；只要 a>=b 就做 a-=b，因此 a 依序為 5、3、1。1<2 時輸出 1，等於 5%2。此流程假設 a 非負且 b 正數。

判 6 判斷：已知 'A' 的 ASCII 為 0x41，則 'm' 的 ASCII 八進制為 155。

答案：T（正確）

'A'=65，'a'=97，'m' 比 'a' 大 12，所以為 109。109=1×64+5×8+5，八進制為 155；C++ 八進制常量可寫 0155。

判 7 判斷：C++ 的位運算符有不同優先級，使用括號能明確指定分組。

答案：T（正確）

先處理一元 ~、！，再移位 <<、>>，再 &、^、 |。寫 (a&1)==1 最清楚；不要省成 a&1==1 後只靠直覺判斷。括號只決定分組，並不能解決所有副作用求值順序問題。

判 8 判斷：下列程式中 x 與 y 相等，因為 char 與 int 可以互換。

答案：F（錯誤）

x 儲存的是字元 '1' 的 ASCII 編碼 49；y 儲存整數 1。比較時 x 轉成 int 仍為 49，兩者不相等。char 是否有符號由實作決定，不能一概說 char 能表示 0 至 255。

判 9 判斷：((0xf0 + 0x15) == 255) 的值為 true。

答案：F（錯誤）

0xF0=240，0x15=21，總和為 261，即 0x105，不是 255。255 的十六進制才是 0xFF。這裡是一般 int 計算，不會因寫了兩位十六進制就截成 8 位。

判 10 判斷：若 int 變數 a 的二進制最低位為 0，則 ((a & 3 & 1) == 0) 為 true。

答案：T（正確）

& 可結合計算，3 & 1 = 1，因此 a & 3 & 1 等於 a & 1，只保留最低位。最低位為 0，整個比較便為 true。

---

04 | 2026 / 09 編程 1：公共二進制位

給 n 個非負整數，2 ≤ n ≤ 20000，0 ≤ a[i] ≤ 10^9。求在所有數中都為 1 的二進制位共有幾個。

解題順序

1. 按位與 & 的某一位為 1，當且僅當兩個運算元該位都為 1。

2. 令 common 為第一個數，再依序 common &= x。處理完所有數後，common 的 1 正是共同位。

3. 反覆檢查 common&1 並右移一位，累加 1 的個數。

樣例推演

13=1101、7=0111、15=1111。逐項 AND：1101 & 0111 = 0101，再 &1111 仍為 0101；它有 2 個 1，所以輸出 2。

常見錯法

- common 不能初始化為 0 再逐項 AND，否則永遠是 0。
- 輸出共同位的數量 2，不是按位與的數值 5。
- 若任一數為 0，共同結果必為 0。

效率｜時間 O(n+log A)，額外空間 O(1)；A 為最大輸入值。

完整 C++11 程式

```
#include <iostream>
using namespace std;
int main() {
    int n;
    unsigned int common, x;
    cin >> n >> common;                    // 第一個數作初值
    for (int i = 1; i < n; i++) {
        cin >> x;
        common &= x;
    }
    int ans = 0;
    while (common > 0) {
        ans += common & 1u;
        common >>= 1;
    }
    cout << ans << '\n';
    return 0;
}
```

檔案：gesp_2026_09_1.cpp。程式存於完整練習包的 code 資料夾。

04 | 2026 / 09 編程 2：分割字串

輸入一行長度不超過 1000 的文字，可含字母、數字和空格，且不以空格結尾。第一次以 1 個空格分割右側文字，第二次以連續 2 個，第三次以連續 3 個，以此類推；只丟棄本次所需的空格。

解題順序

1. 必須 getline 讀完整行，不能用 cin>>s，否則第一個空格後的內容就沒有讀到。

2. 令 start 表示尚未處理文字的起點，本輪需 k 個空格。從 start 掃描，連續空格數達到 k 時立即切割。

3. 切割點 cut=i-k+1；輸出 s[start..cut-1]，再把 start 設為 i+1，恰好跳過本次使用的 k 個空格。

4. 找到後 k 加 1，從新的 start 繼續。找不到時輸出全部剩餘字串並結束。切出空字串也要輸出換行。

樣例推演

以 · 代表空格，只用於展示：AbCd···999。第 1 輪輸出 AbCd，剩 ··999；第 2 輪立刻遇到 2 個空格，因此輸出空行，剩 999；最後輸出 999。實際答案是三行，第二行不能省略。

常見錯法

- 一串空格比 k 長時，只消耗 k 個，其餘留在右側。
- 小於 k 的空格不觸發切割，要保留在本次輸出的文字內。
- 如果完全找不到分隔，原字串原樣輸出。
- 本解法與原卷參考程式等價，但使用下標掃描，避免反覆建立和複製剩餘字串。

效率 | 下標只往右前進，時間 O(L)，字串空間 O(L)，除輸入與輸出子字串外只需 O(1) 狀態。

完整 C++11 程式

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s;
    getline(cin, s);
    int n = (int)s.size(), start = 0;
    for (int k = 1; ; k++) {
        int run = 0, cut = -1;
        for (int i = start; i < n; i++) {
            if (s[i] == ' ') run++;
            else run = 0;
            if (run == k) {
                cut = i - k + 1;          // 分隔空格的起點
                break;
            }
        }
        if (cut == -1) {
            cout << s.substr(start) << '\n';
            break;
        }
        cout << s.substr(start, cut - start) << '\n';
        start = cut + k;                  // 只消耗 k 個空格
    }
    return 0;
}
```

04 | 2026 / 06 編程 1：加密

先輸入 n 個 0..9 的數字，再輸入 10 個數字作為對應表；n 不超過 20000。把每個輸入數字替換成密碼本指定的數字。

解題順序

1. 先把原數列存進 a，因為密碼本在輸入的最後一行。

2. 以 key[0]..key[9] 儲存對應表，key[d] 就是數字 d 加密後的結果。

3. 依原數列順序輸出 key[a[i]]。只做一次映射，不要對剛得到的新數字再映射。

樣例推演

密碼本為 9 0 1 2 3 4 5 6 7 8 時，key[0]=9、key[2]=1。輸入 0 2 0 3 4 1 9，輸出 9 1 9 2 3 0 8。

常見錯法

- 密碼本第一格對應數字 0，不是數字 1。
- 輸出 key[a[i]]，不是 key[i] 或 a[key[i]]。
- 數字之間用空格分隔，末尾輸出換行。

效率｜時間 O(n+10)，額外空間 O(n+10)。

完整 C++11 程式

```
#include <iostream>
using namespace std;
int a[20005], key[10];
int main() {
    int n;
    cin >> n;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int d = 0; d < 10; d++) cin >> key[d];
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << ' ';
        cout << key[a[i]];                // 以原數字查表
    }
    cout << '\n';
    return 0;
}
```

檔案：gesp_2026_06_1.cpp。程式存於完整練習包的 code 資料夾。

04 | 2026 / 06 編程 2：字元轉換

輸入 n 和長度為 n 的字串，字元只可能為大小寫英文字母或數字，n ≤ 1000。大小寫互換，數字改為 *。

解題順序

1. 讀入整個不含空格的字串。

2. 逐字判斷：大寫變小寫，小寫變大寫，其他合法輸入就是數字，輸出 *。

3. 使用 if / else if / else，只讓每個原字元進入一個分支。

樣例推演

aBc98 轉換成 AbC**：a→A、B→b、c→C、9→*、8→*。

常見錯法

• 不要寫成連續兩個獨立 if，否則修改後的字元可能又被改回去。

• 比較字元要用 'A'、'Z'、'a'、'z'；輸出時轉回 char。

• 輸出字元之間沒有空格。

效率 | 時間 O(n)，字串儲存空間 O(n)，額外處理空間 O(1)。

完整 C++11 程式

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    int n;
    string s;
    cin >> n >> s;
    for (int i = 0; i < n; i++) {
        char c = s[i];
        if ('A' <= c && c <= 'Z')
            cout << char(c - 'A' + 'a');
        else if ('a' <= c && c <= 'z')
            cout << char(c - 'a' + 'A');
        else
            cout << '*';                    // 題目保證是數字
    }
    cout << '\n';
    return 0;
}
```

檔案：gesp_2026_06_2.cpp。程式存於完整練習包的 code 資料夾。

04 | 2026 / 03 編程 1：二進制回文串

輸入 1 ≤ n ≤ 100000，計算 1 至 n 中，無前導零的二進制表示為回文的整數數量。

解題順序

1. 枚舉每個候選 x，使用 t=x，避免除法過程改掉 x。

2. 反覆 t%2 取最低位、t/=2 去掉最低位，把位元存入陣列。儲存順序雖然由低到高，仍可判斷是否回文。

3. 比較 a[l] 與 a[r]，由兩端往中間走；任何一對不同即不成立。

4. 每處理新的 x，都重設 len 和 ok；成立才把 ans 加 1。

樣例推演

輸入 15，符合者為 1、3、5、7、9、15；二進制依序為 1、11、101、111、1001、1111，共 6 個。

常見錯法

- 不能固定補成 8 位再判斷，否則 1 會被誤判。
- 本題從 1 開始，不把 0 納入計數。
- 只讀 a[0]..a[len-1]；上一個數留下的其他陣列內容不影響。

效率 | 時間 O(n log n)，額外空間 O(log n)。在 n ≤ 100000 下足夠。

完整 C++11 程式

```
#include <iostream>
using namespace std;
int main() {
    int n, ans = 0;
    cin >> n;
    for (int x = 1; x <= n; x++) {
        int a[32], len = 0;
        int t = x;                          // 用副本做轉換
        while (t > 0) {
            a[len++] = t % 2;               // 存最低位
            t /= 2;
        }
        bool ok = true;
        for (int l = 0, r = len - 1; l < r; l++, r--) {
            if (a[l] != a[r]) {
                ok = false;
                break;
            }
        }
        if (ok) ans++;
    }
    cout << ans << '\n';
    return 0;
}
```

檔案：gesp_2026_03_1.cpp。程式存於完整練習包的 code 資料夾。

04 | 2026 / 03 編程 2：凱撒密碼

給定已知明文、對應密文、待破解密文，三者均為大寫字母且長度不超過 1000。求待破解文字的明文。

解題順序

1. 由已知第一組字元算加密位移 k=(密文[0]-明文[0]+26)%26。題目保證全篇使用同一位移，因此一組已知對應已足夠。

2. 對每個待破解字元 c，先用 c-'A' 轉成 0..25。

3. 解密要向回移 k 格：(c-'A'-k+26)%26，再加 'A' 還原字元。

4. 加 26 是避免負數取模。因 k 已在 0..25，這裡加一次 26 就足夠。

樣例推演

已知 A→D，得到 k=3。密文 WKH 依序往回 3 格得到 THE；密文 ABC 則回到 XYZ。

常見錯法

- 加密加 k，解密減 k，方向不要相反。
- 不能直接 c-k，因為跨過 A 時需回到 Z。
- 本題不處理空格或小寫字母，使用 cin 讀取字串即可。

效率｜設三個字串總長為 L，時間 O(L)，儲存空間 O(L)。

完整 C++11 程式

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string plain, cipher, text;
    cin >> plain >> cipher >> text;
    int k = (cipher[0] - plain[0] + 26) % 26;
    for (int i = 0; i < (int)text.size(); i++) {
        int x = text[i] - 'A';            // 轉成 0..25
        x = (x - k + 26) % 26;            // 逆向位移
        cout << char('A' + x);
    }
    cout << '\n';
    return 0;
}
```

檔案：gesp_2026_03_2.cpp。程式存於完整練習包的 code 資料夾。

05 ｜同型題複習索引

知識點	照片卷題號	三卷對應題號
進制換算 / 數據單位	1、16、22、23、24	03 選6；06 選1/2/3/8、判6；09 選1/2/15
原碼 / 反碼 / 補碼	2、3、4、15、27、28、29、30	03 選5/7；06 選4/10、判1/2/3；09 選4、判1/2/3
位運算與優先級	5、6、17、18、19、25、31、32	03 選2/8/9、判7/10；06 選6/9/11/12、判5；09 選5/6/7/8/10/11/12、判5
字元 / 字串	7、8、9、20、21、33、35、37、38	03 選1/3/10/12/15、判6/8/9；06 選5/7/13/15、判7/10；09 選9/13、判6/7
陣列 / 流程 / 語法	10、11、26、34、39、40	03 選4/11、判1/2/3/4/5；06 選14、判8；09 選3/14、判4/8/10
枚舉 / 模擬	12、13、14、36	03 選13/14；06 判4/9；09 判9

我的錯題記錄

卷別 / 題號	錯因（觀念 / 讀題 / 計算）	一句話改正	重做日期

來源、版本與核對說明

本冊主要依據你提供的兩張照片及三份 PDF。月份採檔案與卷面標示。完整練習包保留原始三份 PDF，供逐頁核對。

GESP_2026-09_C++三級_官方真題.pdf

GESP_2026-06_C++三級_官方真題.pdf

GESP_2026-03_C++三級_官方真題.pdf

另外附上的《數組訂正與二分查找》與《高二考前專題練習_1028》屬補充材料；本次以明確指定的照片 40 題及三份 GESP 卷為範圍，沒有把補充教材混稱為 GESP 原卷。

容易混淆的原卷表述

1. 六月選擇第 4 題：範圍未清楚指定編碼，按給定答案採補碼解讀；詳解指出歧義。
2. 三月選擇第 3 題：字元陣列不同是答案 C，但沒有終止空字元的輸出還涉及未定義行為。
3. 六月選擇第 2 題：約 931 的單位嚴格應為 GiB。
4. 六月選擇第 15 題、九月選擇第 9 題：只看最大字元的判法需合法數碼輸入前提。
5. 九月判斷第 10 題：流程累加到 a[n-1]，題幹卻寫包含 a[n]。

C++ 規則核對資料（公開標準工作草案）

字串 erase
https://eel.is/c++draft/string.erase

範圍 for 與參考
https://eel.is/c++draft/stmt.ranged

字元陣列初始化
https://eel.is/c++draft/dcl.init.string

無先後次序副作用 / 未定義行為
https://eel.is/c++draft/intro.execution

查閱日：2026-11-03。以上為公開草案，僅用於核對列出的語義；涉及不同標準版本的題目，仍以題幹指定的 C++11 及試卷前提處理。

程式驗證

6 份程式均以 g++ -std=c++11 -O2 -Wall -Wextra -pedantic 編譯，並通過 812 組功能測試，包含所有 26 種凱撒位移、回文數上界、隨機映射、共同位元和含空行的字串分割。另核對 3 組有明確定義的位運算算式。這些測試提供實作驗證，不代表官方評測結果。